



UNIVERSIDAD DE DAGUPAN

SCHOOL OF INFORMATION TECHNOLOGY EDUCATION

ITP10 | EVENT-DRIVEN PROGRAMMING

MIDTERM | LABORATORY ACTIVITY 1

Name : _____

Year, Course & blk : _____

Subject : _____

Date : _____

SCORE:

Laboratory Activity #1: Interactive Student Profile Controller

Objective: After completing the laboratory activity, students should be able to select DOM elements using appropriate selector methods, safely read and change element content, manipulate attributes and custom data attributes, manage visual states with classList, and apply defensive DOM programming practices.

Activity Overview: Students will develop an Interactive Student Profile Controller. A control panel will allow the user to update a student profile card without reloading the page. The application must use DOM selection and manipulation techniques from Module 4 and must treat all user-entered text as untrusted content.

Required Project Structure

```
student-profile-controller/  
├── index.html  
├── css/  
│   └── style.css  
├── js/  
│   └── app.js
```

Application Requirements

1. Initial Student Profile

- Create a visible profile card with the initial values shown below. Keep the required IDs, classes, data attributes, and initial text exactly as specified so automated tests can evaluate the page consistently.
- Initial student name: Maria Santos
- Initial program: BS Information Technology
- Initial year level: 3rd Year
- Initial status: Active
- The profile card must start with data-student-id="2026-001" and data-status="active".

```
<article id="profileCard" class="profile-card active" data-student-id="2026-001" data-status="active">  
  <h2 id="profileName">Maria Santos</h2>  
  <p id="profileProgram">BS Information Technology</p>  
  <p id="profileYear">3rd Year</p>  
  <p id="profileStatus">Active</p>  
  <div id="detailsPanel" class="details-panel">  
    <p id="studentIdDisplay">Student ID: 2026-001</p>  
  </div>  
</article>
```

2. DOM Selection Requirements

- Use document.getElementById() for at least four elements with unique IDs.
- Use document.querySelector() for at least one element or CSS selector.
- Store frequently used DOM elements in variables instead of repeatedly querying the same element.
- Before modifying a selected element, verify that the element exists. Avoid code that attempts to modify null.

3. Required JavaScript Functions

- isValidStudentName(name) — return true when the trimmed name contains at least 2 characters; otherwise return false.
- formatStudentStatus(status) — return "Active" for the value "active" and "Inactive" for the value "inactive".
- updateProfile() — validate the form, then update the profile card using the current control values.
- setStatus(status) — update the profile status text, data-status value, and active/inactive CSS classes.
- toggleDetails() — show or hide the details panel using classList.toggle().
- toggleTheme() — toggle the class dark-theme on document.body.
- resetProfile() — restore the exact initial profile data, status, controls, message, details visibility, and theme.

4. Safe Content and State Rules

- All user-entered values must be written to the page using textContent. Do not use innerHTML for name, program, year level, status, messages, or any other user-controlled value.
- Use profileCard.dataset.studentId to read the initial student ID and profileCard.dataset.status to manage the status value.
- Use classList.add(), classList.remove(), or classList.toggle() to manage active, inactive, hidden, and dark-theme states. Do not overwrite the complete className string.
- Inline styles may be used only for a value that must be calculated dynamically. Regular visual states must be handled through CSS classes.
- When the name is invalid, do not update the profile. Display the exact message "Student name is required" in #formMessage.

5. User Interface and Required IDs

Interface Element	Required ID / Class	Required Text / Value	Notes
Application title	—	Interactive Student Profile Controller	Visible title
Name input	nameInput	placeholder="Enter student name"	Text input
Program select	programInput	BS Information Technology; BS Computer Science; BS Information Systems	Use exact option values
Year-level select	yearInput	1st Year; 2nd Year; 3rd Year; 4th Year	Use exact option values
Status select	statusInput	active; inactive	Use lowercase option values
Update button	updateBtn	Update Profile	Button
Toggle details button	toggleDetailsBtn	Toggle Details	Button
Theme button	themeBtn	Toggle Theme	Button
Reset button	resetBtn	Reset	Button
Profile card	profileCard / .profile-card	—	Main profile container
Profile name	profileName	Maria Santos	Initial text
Program text	profileProgram	BS Information Technology	Initial text
Year text	profileYear	3rd Year	Initial text
Status text	profileStatus	Active	Initial text
Details panel	detailsPanel / .details-panel	—	Can be hidden
Student ID display	studentIdDisplay	Student ID: 2026-001	Read from dataset
Form message	formMessage	—	Validation/success message

Expected Behavior

- On initial page load, the exact initial student profile is visible and the status is Active.
- Clicking Update Profile with valid values changes only the corresponding text and status state; the page must not reload.
- When status is inactive, #profileCard must have class inactive, must not have class active, and dataset.status must equal "inactive".
- Toggle Details must hide/show #detailsPanel using the hidden class.
- Toggle Theme must add/remove dark-theme on the body element.
- Reset must restore the exact initial values, remove dark-theme, show the details panel, clear #formMessage, and set the form controls back to the initial values.

Automated-Grading Compatibility

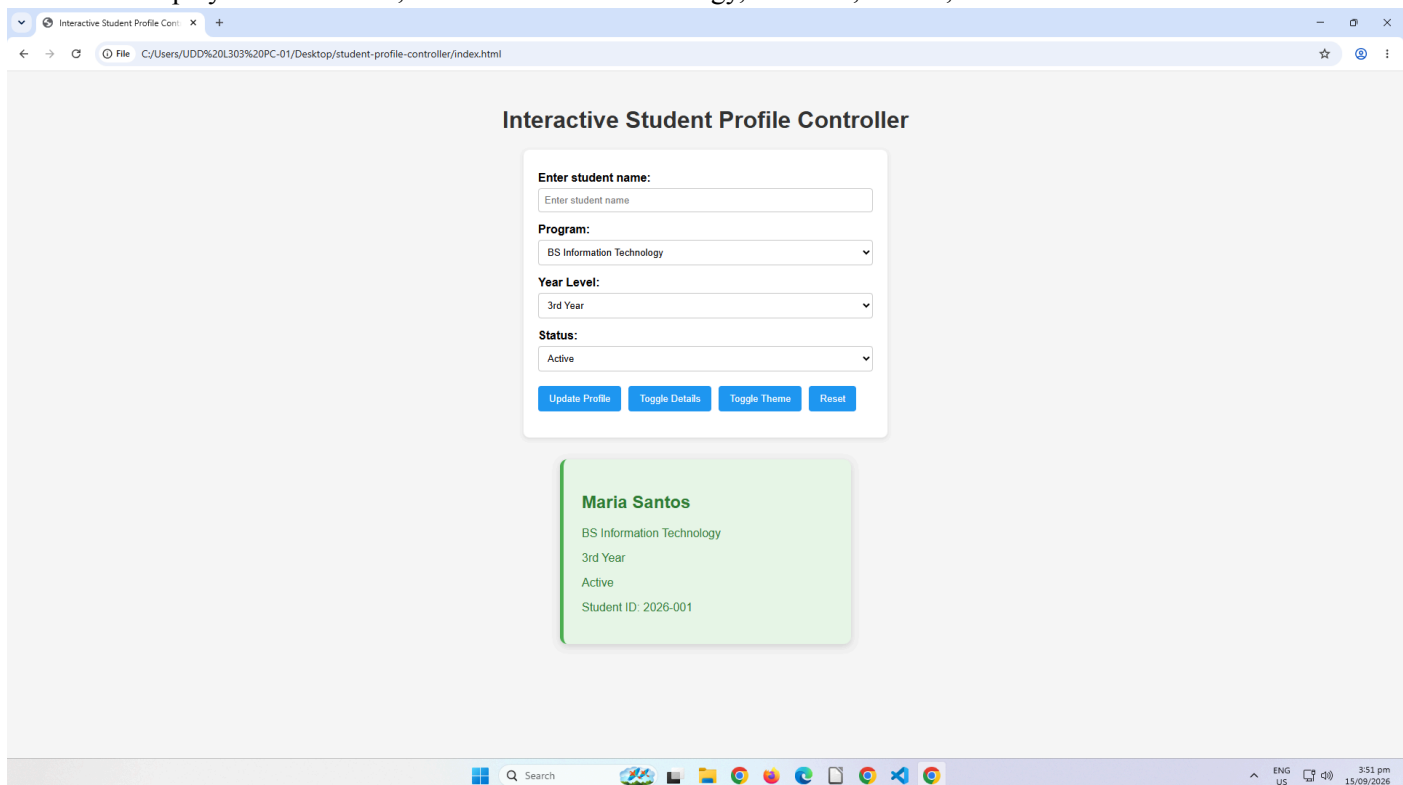
- Do not rename required files, function names, IDs, classes, option values, data attributes, button text, or required messages.
- Do not hard-code alternate results specifically for the tests. Functions must work using the current DOM and user input.
- Do not use innerHTML to insert user-controlled content.
- Keep JavaScript in js/app.js and connect it using `<script src="js/app.js" defer></script>` or an equivalent deferred script placement.
- Students may design the interface freely as long as the required selectors, content, states, and behavior remain intact.

General Procedure

1. Create the required folders and files.
2. Build the initial profile card and the update controls in index.html using the exact required IDs and values.
3. Create readable CSS for the profile card, controls, active/inactive state, hidden state, and dark theme.
4. Select and store the required DOM elements in app.js.
5. Implement `isValidStudentName()` and `formatStudentStatus()`.
6. Implement `updateProfile()` using `textContent` for all user-entered text.
7. Implement dataset and classList state changes in `setStatus()`.
8. Implement Toggle Details, Toggle Theme, and Reset behaviors.
9. Test valid and invalid input, status changes, class changes, data attributes, theme state, and reset behavior.
10. Check the browser console and correct all errors before submission.

Testing Requirements

1. Initial load displays Maria Santos, BS Information Technology, 3rd Year, Active, and Student ID: 2026-001.



2. Updating the name to "Ana Cruz" displays exactly Ana Cruz in #profileName.

The screenshot shows a web browser window with the title "Interactive Student Profile Controller". The address bar shows the file path: "C:/Users/UDL303%20PC-01/Desktop/student-profile-controller/index.html". The main content area displays a form titled "Interactive Student Profile Controller". The form has the following fields and buttons:

- Enter student name:** A text input field containing "Ana Cruz".
- Program:** A dropdown menu showing "BS Information Technology".
- Year Level:** A dropdown menu showing "3rd Year".
- Status:** A dropdown menu showing "Active".
- Buttons:** "Update Profile", "Toggle Details", "Toggle Theme", and "Reset".

Below the form, a green profile card displays the updated information:

- Ana Cruz**
- BS Information Technology
- 3rd Year
- Active
- Student ID: 2026-001

The Windows taskbar at the bottom shows the search bar, task view button, and several application icons. The system tray on the right shows the language "ENG US", a network icon, and the time "3:51 pm 15/09/2026".

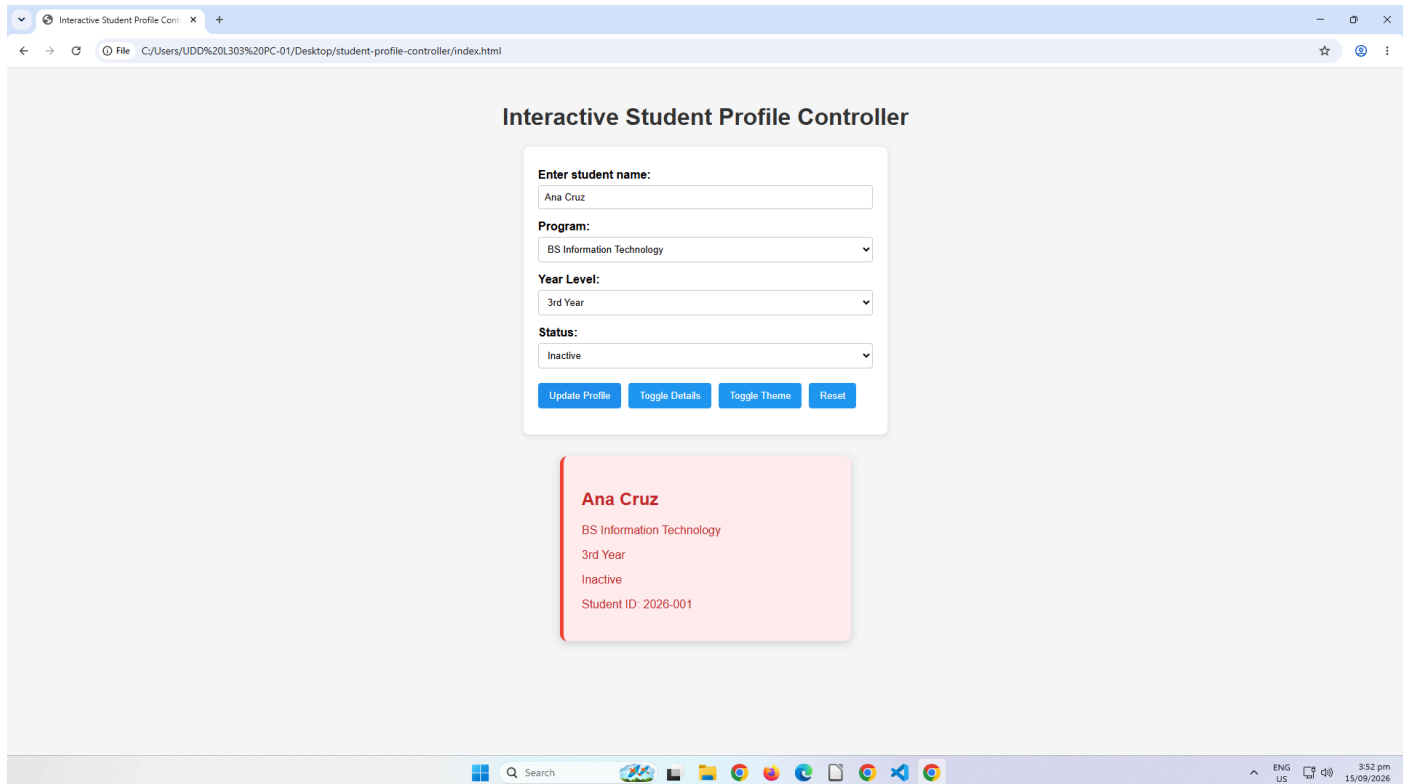
3. Submitting a blank or whitespace-only name does not update the profile and displays "Student name is required".

The screenshot shows the same web browser window as the previous one. The form titled "Interactive Student Profile Controller" is displayed. The "Enter student name:" field is empty. Below the form, a red error message is visible: "Student name is required". The profile card below the form still displays the previous information:

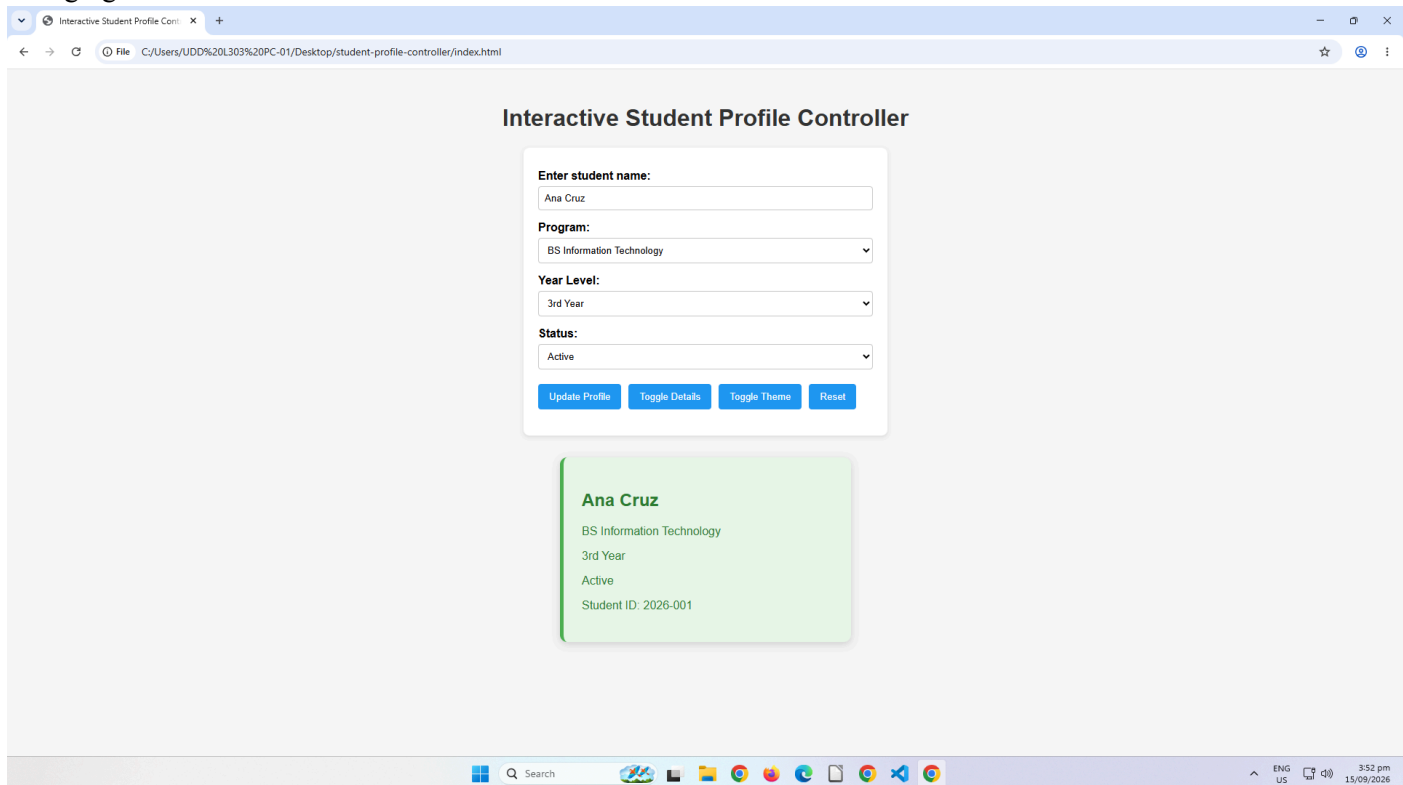
- Ana Cruz**
- BS Information Technology
- 3rd Year
- Active
- Student ID: 2026-001

The Windows taskbar and system tray are identical to the previous screenshot.

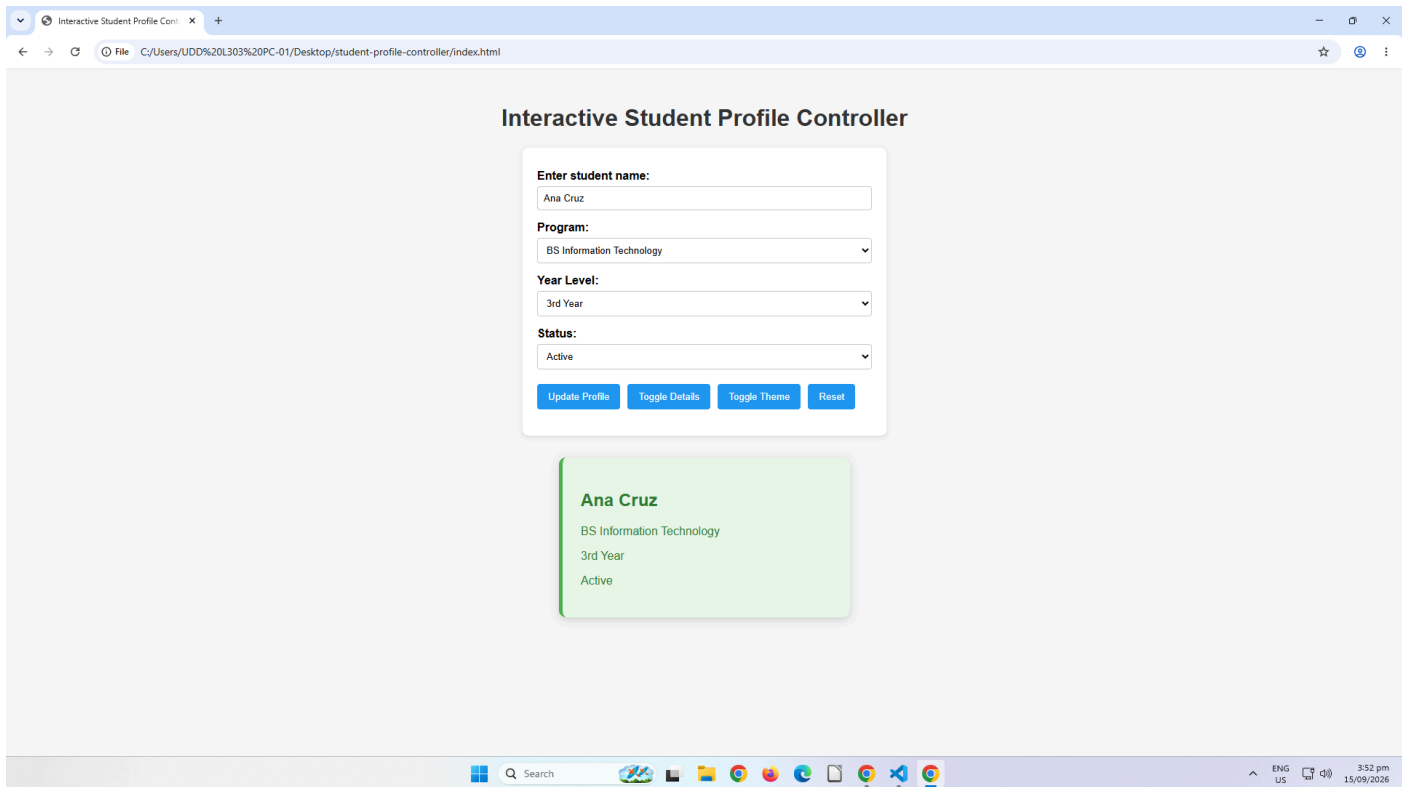
4. Changing status to inactive updates #profileStatus to Inactive, dataset.status to inactive, and the card state class to inactive.



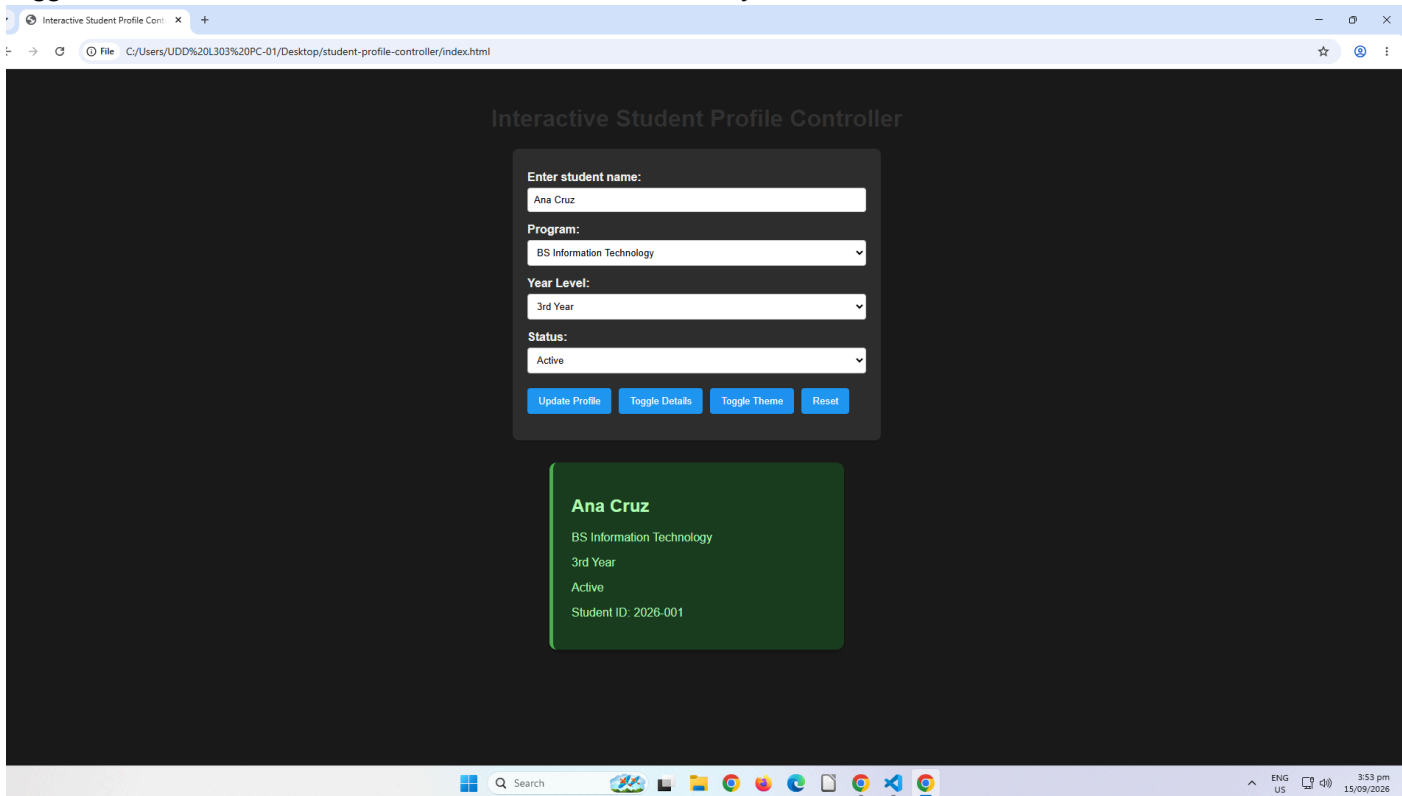
5. Changing status back to active restores the Active text and active class.



6. Toggle Details hides and then restores the details panel using the hidden class.

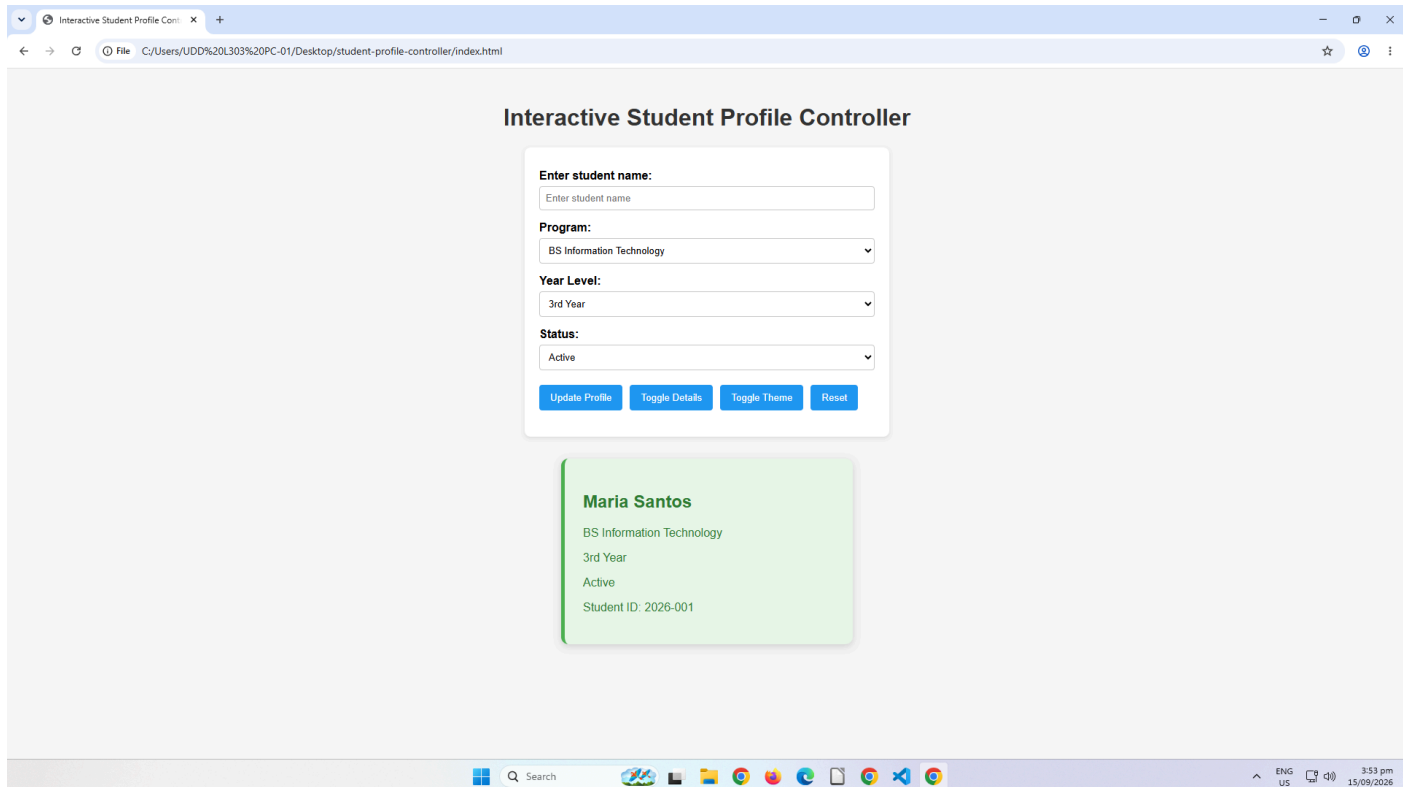


7. Toggle Theme adds and removes dark-theme on document.body.



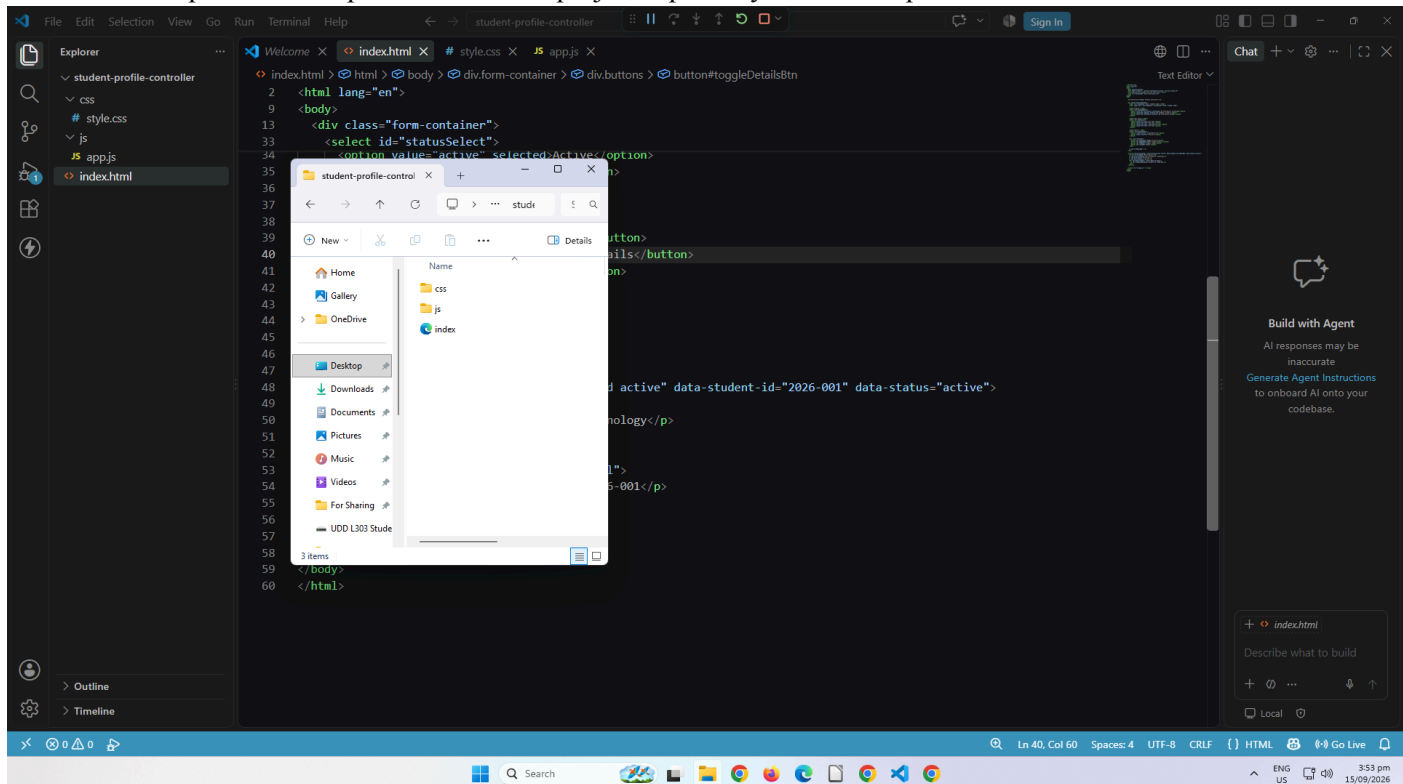
8. The student ID displayed on the page comes from data-student-id.
9. User-entered text containing characters such as <, >, and & is displayed as text and is not interpreted as HTML.

10. Reset restores all exact initial values and states.

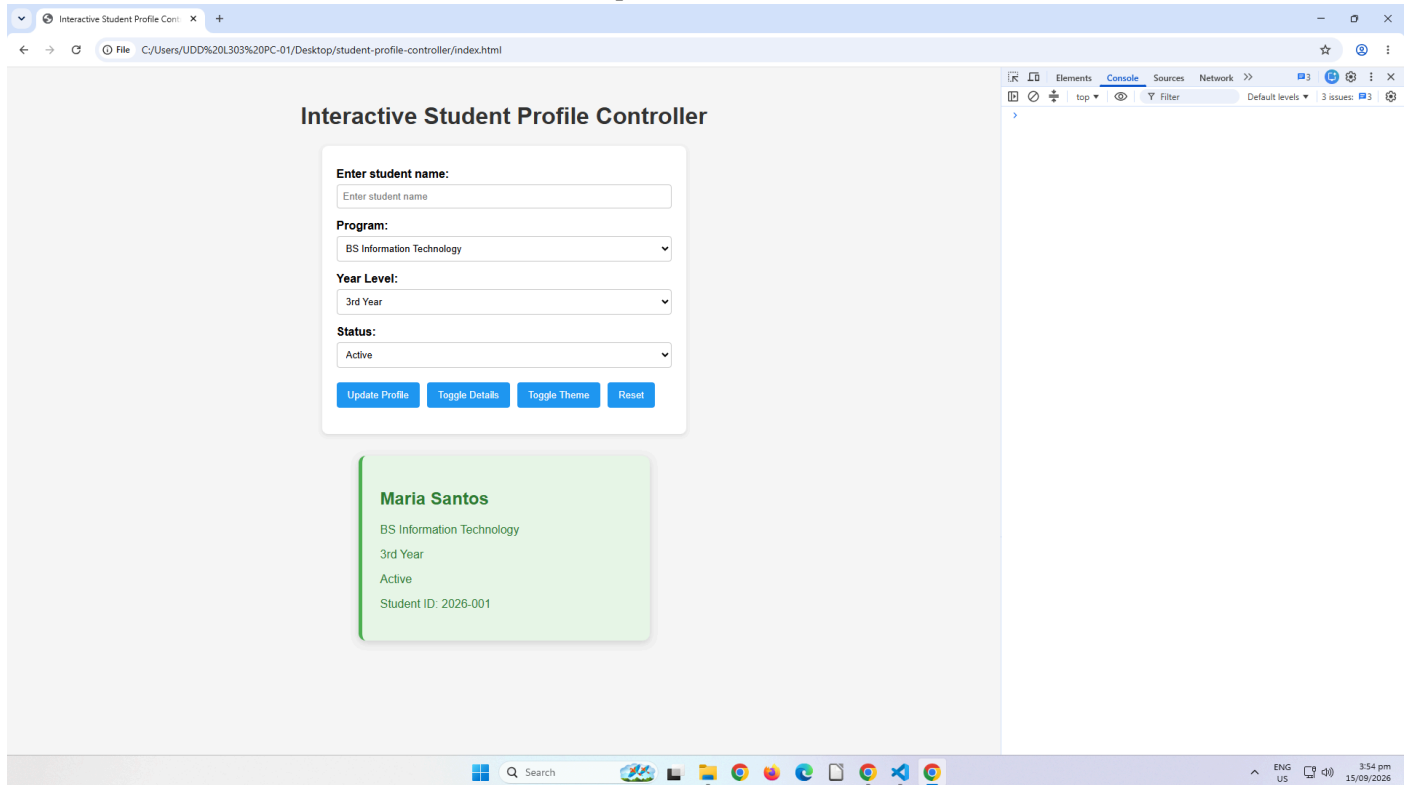


Submission Requirements

- Submit the complete student-profile-controller project/repository with the required folder structure.



- Make sure the browser console contains no JavaScript errors.



- Include the screenshots or explanations required by the instructor/classroom submission page.

Grading Rubric

Criterion	Excellent	Proficient	Developing	Beginning
Application Functionality (30 points)	All required features work correctly. (27–30)	Most features work with minor errors. (22–26)	Some features work, but several errors are present. (16–21)	The application is incomplete or does not function properly. (0–15)
Modular Organization (25 points)	Code is properly divided into data, utility, display, and main modules with correct imports and exports. 23–25	Modules are properly used with minor organizational problems. 18–22	Modules are present, but responsibilities or connections are unclear. 13–17	Code is mainly placed in one file, or modules do not work. 0–12
Use of JavaScript Concepts (20 points)	Correctly uses functions, arrays, objects, callbacks, array methods, and destructuring. 18–20	Uses most required concepts correctly. 14–17	Uses some concepts but with noticeable errors. 10–13	Required JavaScript concepts are mostly missing or incorrectly applied. 0–9
User Interface and Output (10 points)	Interface is clear, organized, readable, and displays complete information. 9–10	Interface is usable with minor presentation issues. 7–8	Interface or displayed information is incomplete. 5–6	Interface is difficult to use or output is unclear. 0–4
Code Quality (10 points)	Code is readable, properly indented, meaningfully named, and free from unnecessary repetition. 9–10	Code is generally readable with minor inconsistencies. 7–8	Code has inconsistent formatting or unnecessary repetition. 5–6	Code is difficult to understand or poorly organized. 0–4
Testing and Submission (5 points)	All required files, screenshots, explanations, and test results are complete. 5	One minor requirement is missing. 4	Several submission requirements are incomplete. 2–3	Submission is substantially incomplete. 0–1
Total				100 points

PREPARED BY:

ROCHELLE S. LANTO, MIT & JELSON V. LANTO, MIT
SITE, FACULTY